

Trabalho de Inteligência Artificial

Disciplina:

Inteligência Artificial

Integrantes do Grupo:

Felipe Guerzoni Martins Flôres Maia

Felipe Rivetti Mizher

Marcos Paulo Miranda Pereira

Matheus Felipe Cavalcanti Xavier

Professor:

Gabriel Pereira de Oliveira

Pontifícia Universidade Católica de Minas Gerais

Junho - 2026

Introdução:

O 8-Puzzle é um jogo composto por um tabuleiro 3×3 contendo 8 peças numeradas de 1 a 8 e um espaço vazio. O objetivo é mover as peças até alcançar uma configuração final desejada.

O problema é modelado como um problema de busca em espaço de estados. Cada posição do tabuleiro representa um estado, e os movimentos do espaço vazio representam as ações que levam de um estado para outro. O objetivo é encontrar uma sequência de movimentos que transforme o estado inicial no estado objetivo com o menor custo possível. Para isso, podem ser utilizados algoritmos de busca como BFS, DFS, Busca Gulosa e A*.

Modelagem:

O problema do **8-Puzzle** foi modelado como um problema de busca em espaço de estados. Cada estado representa uma configuração do tabuleiro 3×3, contendo oito peças numeradas e um espaço vazio, armazenados em uma tupla com nove posições. O estado inicial é informado pelo usuário, enquanto o estado objetivo corresponde à configuração (1, 2, 3, 4, 5, 6, 7, 8, 0), onde o valor **0** representa o espaço vazio.

As ações possíveis consistem em movimentar o espaço vazio para **cima, baixo, esquerda ou direita**, respeitando os limites do tabuleiro. Cada movimento gera um novo estado sucessor e possui custo unitário, permitindo que algoritmos como Busca em Largura, Busca Uniforme, Busca Gulosa e A* explorem o espaço de estados. Além disso, foi utilizada uma classe para representar cada nó da árvore de busca, armazenando informações como estado atual, nó pai, movimento realizado, custo acumulado e profundidade.

A implementação foi organizada de forma modular, separando a representação dos estados, a geração dos sucessores, as heurísticas e os algoritmos de busca em arquivos distintos. Essa organização facilitou a reutilização do código e permitiu que todos os algoritmos utilizassem a mesma estrutura de dados, diferenciando-se apenas pela estratégia empregada para selecionar o próximo nó a ser expandido.

Descrição dos Algoritmos:

Busca A*

Funcionamento:

O algoritmo A* combina o custo já percorrido com uma estimativa do custo restante até o objetivo.

Para isso utiliza a função: $F(n) = g(n) + h(n)$.

Completeness:

Sim, desde que a heurística seja admissível.

Optimality:

Sim. Quando a heurística é admissível e consistente, o A* sempre encontra a solução ótima.

Complexidade tempo/espço:

- Tempo: $O(b^d)$
- Espaço: $O(b^d)$

Buscar em Largura (BFS)

Funcionamento:

A Busca em Largura (BFS) realiza uma busca dos estados por níveis. Primeiro visita todos os nós a uma determinada profundidade antes de avançar para a próxima. Utiliza uma fila (FIFO) para armazenar os nós a serem expandidos.

Completeness:

A BFS é completa, pois sempre encontrará uma solução se ela existir em um espaço onde a busca é finita.

Optimality:

Quando todos os caminhos possuem o mesmo custo, como ocorre no problema do 8-Puzzle, a primeira solução encontrada pela Busca em Largura (BFS) será aquela com a menor profundidade. A BFS garante encontrar a solução de menor custo, sendo, portanto, um algoritmo ótimo nesse contexto.

Complexidade tempo/espço:

- Tempo: $O(b^d)$
- Espaço: $O(b^d)$

Busca em Profundidade (DFS)

Funcionamento:

A Busca em Profundidade (DFS) expande sempre o nó mais profundo da árvore antes de explorar outros caminhos. Utiliza uma pilha (FILO) ou recursão para controlar a expansão.

Completeness:

Não em todos os casos. Em espaços de busca muito grandes ou infinitos, pode ficar presa em um ramo e não encontrar a solução para o problema.

Optimality:

Não é um algoritmo ótimo, pois a primeira solução encontrada não pode ser a melhor nem a de menor custo para o programa.

Complexidade tempo/espço:

- Tempo: $O(b^m)$
- Espaço: $O(b \cdot m)$

Busca Uniforme (UCS)

Funcionamento:

Busca de Custo Uniforme expande sempre o nó com menor custo acumulado desde o estado inicial. Utiliza uma fila de prioridade ordenada pelo valor de $g(n)$, que representa o custo do caminho percorrido.

Completeness:

Sim, com a condição de que todos os custos sejam positivos.

Optimality:

Sempre encontra o caminho de menor custo até o objetivo.

Complexidade tempo/espço:

- Tempo: Exponencial no pior caso
- Espaço: Exponencial no pior caso

Busca Gulosa

Funcionamento:

A Busca Gulosa utiliza apenas a heurística $h(n)$ para decidir qual nó será o próximo a expandir. Sempre escolhe o estado que aparenta estar mais próximo da solução.

Completeness:

Dependendo da heurística utilizada, pode não encontrar uma solução para o problema.

Optimality:

Pode encontrar rapidamente uma solução, mas não quer dizer que será sempre a melhor solução.

Complexidade tempo/espço:

- Tempo: $O(b^m)$
- Espaço: $O(b^m)$

Heurísticas para o A*:

1. Heurística da Distância Euclidiana

A heurística da **Distância Euclidiana** estima o custo restante até o estado objetivo calculando a distância em linha reta entre a posição atual de cada peça e sua posição final no tabuleiro. Para cada peça (desconsiderando o espaço vazio), é aplicada a fórmula da distância euclidiana:

$$D = \text{raiz}((x1 - x2)^2 + (y1 - y2)^2)$$

O valor da heurística corresponde à soma das distâncias de todas as peças. Embora forneça uma estimativa da proximidade do objetivo, essa heurística considera movimentos em linha reta, enquanto no 8-Puzzle as peças só podem se mover nas direções horizontal e vertical. Por esse motivo, ela costuma ser menos precisa que a distância de Manhattan, mas ainda é útil para orientar a busca.

2. Heurística de Peças Fora do Lugar

A heurística de **Peças Fora do Lugar** (Misplaced Tiles) é uma das mais simples utilizadas no problema do 8-Puzzle. Ela consiste em contar quantas peças não estão na posição correta em relação ao estado objetivo, ignorando o espaço vazio.

Cada peça fora de sua posição contribui com o valor 1 para a heurística, independentemente da distância que ela esteja do local correto. Assim, quanto maior o número de peças fora do lugar, maior será o valor da heurística e, consequentemente, mais distante o estado é considerado da solução.

Apesar de possuir baixo custo computacional, essa heurística fornece pouca informação sobre o problema, pois trata da mesma forma uma peça que está apenas uma posição distante e outra que está muito longe da posição correta.

3. Heurística da Distância de Manhattan

A heurística da **Distância de Manhattan** estima o custo restante até o estado objetivo somando, para cada peça, o número mínimo de movimentos horizontais e verticais necessários para alcançar sua posição correta. Para isso, utiliza a seguinte expressão:

$$D = |x1 - x2| + |y1 - y2|$$

O valor total da heurística é obtido pela soma das distâncias de Manhattan de todas as peças, desconsiderando o espaço vazio.

Essa é a heurística mais utilizada no problema do 8-Puzzle, pois representa com maior fidelidade os movimentos permitidos pelo jogo, nos quais as peças só podem ser deslocadas para cima, baixo, esquerda ou direita. Além disso, é uma heurística **admissível e consistente**, garantindo que o algoritmo A* encontre uma solução ótima quando utilizada.

Resultados experimentais:

Tabuleiro base utilizado:

1 2 3

5 6 4

7 0 8

Algoritmos	Tempo de execução	Número de nós expandidos	Profundidade do nó
BFS	0.05215	8446	15
DFS	0.64413	146208	109661
Uniforme	0.03945	8446	15
Gulosa (Fora do lugar)	0.0015	106	15
Gulosa (Manhattan)	0.00090	38	15
Gulosa (Euclidiana)	0.00251	43	15
A* (Fora do lugar)	0.00686	384	15
A* (Manhattan)	0.00423	171	15
A* (Euclidiana)	0.00756	194	15

Análise comparativa:

Os resultados obtidos demonstram que os algoritmos apresentaram desempenhos bastante distintos. A **Busca em Profundidade (DFS)** foi a menos eficiente, expandindo **146.208 nós** e encontrando uma solução com profundidade muito elevada, evidenciando sua tendência de explorar caminhos longos antes de retroceder. Já a **Busca em Largura (BFS)** e a **Busca de Custo Uniforme** apresentaram o mesmo comportamento, expandindo **8.446 nós** e encontrando a solução ótima com profundidade **15**, o que era esperado devido ao custo uniforme das ações no problema do 8-Puzzle.

Entre os algoritmos heurísticos, a **Busca Gulosa utilizando a heurística da Distância de Manhattan** apresentou o melhor desempenho, expandindo apenas **38 nós** e obtendo o menor tempo de execução. No algoritmo **A***, a heurística de **Manhattan** também foi a mais eficiente, expandindo **171 nós**, seguida pelas heurísticas Euclidiana e Peças Fora do Lugar. Assim, conclui-se que a heurística da Distância de Manhattan é a mais adequada para o 8-Puzzle, e que o **A*** representa o melhor equilíbrio entre eficiência e garantia de encontrar a solução ótima.

Diário de decisões:

A maior dificuldade técnica encontrada durante o desenvolvimento foi a implementação dos algoritmos de busca heurística, principalmente o **A***, devido à necessidade de combinar corretamente o custo acumulado do caminho ($g(n)$) com o valor das heurísticas ($h(n)$). Também houve um desafio na implementação das três heurísticas (Peças Fora do Lugar, Distância Euclidiana e Distância de Manhattan), garantindo que todas funcionassem corretamente sobre a mesma estrutura do problema. Essa dificuldade foi resolvida modularizando o projeto, separando a representação do estado, a geração de sucessores e as heurísticas em arquivos independentes, facilitando os testes e a reutilização do código.

Durante o desenvolvimento, algumas abordagens precisaram ser revistas. Inicialmente, havia repetição de código entre os algoritmos de busca e problemas na manipulação dos estados visitados, o que dificultava a manutenção. Após a reorganização da estrutura do projeto, foi possível compartilhar a mesma representação do tabuleiro entre todos os algoritmos. Além disso, os testes mostraram que diferentes heurísticas produzem desempenhos bastante distintos: a heurística **Peças Fora do Lugar** expandiu mais nós, enquanto a **Distância de Manhattan** apresentou os melhores resultados, confirmando sua maior eficiência para o problema do 8-Puzzle.

Caso houvesse mais tempo, seria interessante implementar heurísticas mais sofisticadas, como **Linear Conflict** e **Pattern Databases**, para reduzir ainda mais o número de nós expandidos pelo algoritmo A*. Também seria interessante desenvolver uma interface gráfica para visualizar a execução dos algoritmos passo a passo, além de realizar testes com um número maior de instâncias do problema e coletar métricas adicionais, como consumo de memória e desempenho em diferentes níveis de dificuldade. Essas melhorias tornariam a aplicação mais completa e permitiriam uma comparação ainda mais detalhada entre os algoritmos implementados.

Link para o repositório no GitHub:

<https://github.com/mioj0kt/8puzzle-search-ai/tree/main>